

Analizador lógico

Torres Freddy 1401641, Pérez Andrés 1401638
{est.freddy.torres, est.andresf.perezl}@unimilitar.edu.co
Docente: José rúeles

Resumen — En esta práctica se realizó el análisis de una comunicación serial UART mediante una Raspberry Pi Pico 2W y un analizador lógico, utilizando el software Logic 2 para la captura y decodificación de las señales digitales. Se estudió la relación existente entre la tasa de transmisión, el tiempo de bit y la frecuencia de muestreo del instrumento. Inicialmente se configuró una comunicación UART de 9600 baudios, 8 bits de datos, sin paridad y un bit de parada, con transmisión de caracteres ASCII a través del pin TX de la Raspberry Pi Pico 2W. Posteriormente se analizaron tramas seriales en formatos binario, decimal, hexadecimal y ASCII, y se transmitió el mensaje UMNG_2026_LIDER_EN_TELECOMUNICACIONES.

En la segunda parte se estudió experimentalmente el efecto de modificar la frecuencia de muestreo del analizador lógico sobre la determinación del tiempo de bit. Se analizaron las tasas de transmisión de 1200, 9600, 57600 y 115200 baudios, empleando frecuencias de muestreo entre 25 kS/s y 24 MS/s. Los resultados permitieron comprobar que una mayor frecuencia de muestreo proporciona un mayor número de muestras por bit y, por tanto, una mejor resolución temporal de la señal. Se utilizó como criterio práctico un mínimo de 10 muestras por bit para evaluar la calidad del análisis. Los valores experimentales presentaron pequeñas diferencias respecto de los valores teóricos, debido al proceso de cuantización temporal y a la resolución limitada de las mediciones realizadas con el analizador lógico.

Abstract — This laboratory practice involved the analysis of a UART serial communication using a Raspberry Pi Pico 2W and a logic analyzer with Logic 2 software. The relationship between transmission rate, bit time, and sampling frequency was studied. Initially, a UART communication was configured at 9600 baud, with 8 data bits, no parity, and one stop bit. ASCII characters were transmitted through the TX pin of the Raspberry Pi Pico 2W and the resulting digital waveform was captured and decoded using the logic analyzer.

The serial frames were analyzed in binary, decimal, hexadecimal, and ASCII formats. The message UMNG_2026_LIDER_EN_TELECOMUNICACIONES was also transmitted and analyzed. Subsequently, the effect of the analyzer sampling frequency on bit-time measurement was experimentally evaluated for transmission rates of 1200, 9600, 57600, and 115200 baud, using sampling frequencies from 25 kS/s to 24 MS/s.

The results showed that increasing the sampling frequency provides more samples per bit and therefore improves the temporal resolution of the captured signal. A practical criterion of at least 10 samples per bit was used to evaluate the measurement conditions. The experimental results showed small differences from theoretical values, which are associated with temporal quantization and the finite resolution of the measurement instrument.

Palabras clave — UART, analizador lógico, frecuencia de muestreo, tiempo de bit, Raspberry Pi Pico 2W, Logic 2, comunicaciones digitales

Keywords — UART, logic analyzer, sampling frequency, bit time, Raspberry Pi Pico 2W, Logic 2, digital communications.

I. INTRODUCCIÓN

Las comunicaciones seriales constituyen uno de los mecanismos fundamentales para el intercambio de información entre dispositivos electrónicos. Entre los protocolos de comunicación serial se encuentra UART, ampliamente utilizado debido a su simplicidad y facilidad de implementación. En una comunicación UART, la información se transmite de manera secuencial mediante una línea de transmisión y cada carácter se estructura mediante bits de inicio, bits de datos, un posible bit de paridad y uno o más bits de parada.

El análisis de este tipo de señales permite comprender experimentalmente cómo se representa la información digital en el dominio temporal. Para ello, los analizadores lógicos constituyen una herramienta importante, ya que permiten capturar señales digitales y visualizar sus cambios de estado en función del tiempo. Sin embargo, la calidad de la medición depende de parámetros como la frecuencia de muestreo utilizada durante la captura.

En esta práctica se empleó una Raspberry Pi Pico 2W para generar una señal UART y un analizador lógico para capturarla y decodificarla mediante Logic 2. El objetivo principal fue comprobar la relación entre la tasa de transmisión y el tiempo de bit, además de estudiar experimentalmente cómo la frecuencia de muestreo afecta la resolución temporal y la cantidad de muestras disponibles para representar cada bit. Estos objetivos corresponden a los planteados en la guía de laboratorio.

II. MARCO TEÓRICO

Comunicación UART.

UART, acrónimo de *Universal Asynchronous Receiver/Transmitter*, es un sistema de comunicación serial asíncrona en el que no existe una señal de reloj compartida entre

el transmisor y el receptor. La sincronización se obtiene a partir de la estructura de cada trama.

- Una trama UART convencional puede estar constituida por:
- Un bit de inicio.
- Un conjunto de bits de datos.
- Un bit de paridad, cuando se utiliza.
- Uno o más bits de parada.

En esta práctica se trabajó principalmente con ocho bits de datos y un bit de parada. En la primera etapa se utilizó comunicación sin paridad, mientras que para el análisis de sensibilidad indicado en la guía se empleó paridad impar.

La tasa de transmisión se expresa en baudios. Para una comunicación de un bit por símbolo, la duración nominal de cada bit puede determinarse mediante:

$$T_b = \frac{1}{f_b}$$

UART de la Raspberry Pi Pico 2W y RS232.

Es importante diferenciar una señal UART de nivel lógico de una interfaz RS232 física.

La Raspberry Pi Pico 2W entrega mediante su pin TX una señal UART de nivel lógico de aproximadamente 0 a 3,3 V. Por tanto, al conectar directamente este pin al analizador lógico se está estudiando una señal UART de 3,3 V y no una señal RS232 clásica con niveles eléctricos positivos y negativos.

Para analizar físicamente una interfaz RS232 sería necesario utilizar un transceptor que realizara la conversión entre los niveles lógicos de la UART y los niveles eléctricos propios de RS232.

En la práctica se conectó el canal del analizador lógico al pin TX de la Raspberry Pi Pico 2W, utilizando la referencia de tierra común.

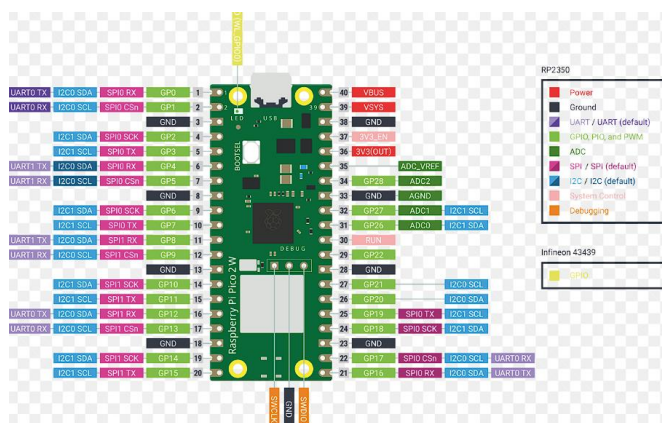


Ilustración 1. datasheet de raspberry pi pico 2w

Protocolo UART (Universal Asynchronous Receiver-Transmitter).

Es un sistema de comunicación serial asíncrona ampliamente utilizado en sistemas embebidos y microprocesadores para el intercambio de datos entre dispositivos electrónicos. Su

funcionamiento se basa en la transmisión y recepción de información bit a bit mediante dos líneas principales: transmisión (TX) y recepción (RX). A diferencia de otros protocolos de comunicación, UART no requiere una señal de reloj compartida, ya que la sincronización se realiza configurando previamente la misma velocidad de transmisión o baudrate en ambos dispositivos. [6]

La comunicación UART es ampliamente utilizada en la Raspberry Pi para establecer conexión con microcontroladores, módulos GPS, dispositivos Bluetooth, sensores y otros sistemas electrónicos. Este protocolo permite enviar y recibir datos de manera sencilla y eficiente, siendo especialmente útil en aplicaciones de monitoreo, automatización y depuración de sistemas. En la Raspberry Pi, la interfaz UART puede utilizarse mediante los pines GPIO específicos para transmisión y recepción serial, facilitando la integración con dispositivos externos. [7]

Otra característica importante del protocolo UART es su simplicidad de implementación tanto en hardware como en software. Mediante lenguajes de programación como Python y librerías como serial o pySerial, es posible controlar la transmisión de datos y establecer comunicación bidireccional entre diferentes dispositivos.

Funcionamiento del analizador lógico.

Un analizador lógico captura señales digitales y determina el estado lógico de una entrada en función de un umbral de tensión. A diferencia de un osciloscopio, su objetivo principal es registrar los estados digitales y sus cambios a lo largo del tiempo.

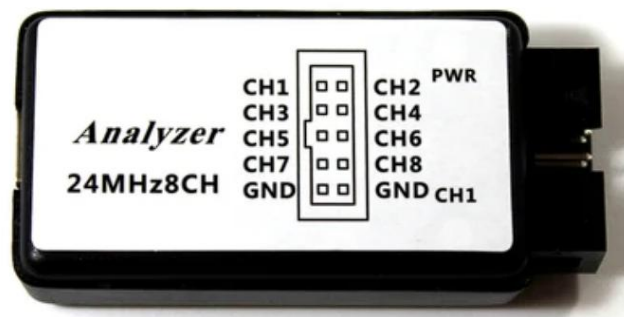


Ilustración 2. analizador lógico de 8 canales

La frecuencia de muestreo (fs) determina la cantidad de muestras obtenidas por unidad de tiempo. La separación temporal entre muestras está dada por:

$$T_s = \frac{1}{f_s}$$

La guía establece como criterio práctico para este laboratorio disponer de **al menos 10 muestras por bit**. Este criterio permite evaluar de forma sencilla si la frecuencia de muestreo

proporciona una resolución temporal suficiente para el análisis de la señal.

El número de muestras por bit se calcula mediante:

$$N = \frac{fs}{fb}$$

Es importante señalar que esta condición corresponde a una regla práctica utilizada para el laboratorio y no constituye una condición absoluta para que cualquier decodificador UART funcione correctamente.

Cuantización temporal.

El analizador lógico solamente dispone de información en los instantes específicos en los que realiza las muestras. Por esta razón, el instante real de una transición puede quedar localizado únicamente dentro del intervalo existente entre muestras.

La guía establece que la incertidumbre temporal asociada a este proceso puede considerarse del orden de un período de muestreo:

$$Ts = \frac{1}{fs}$$

Por esta razón, la incertidumbre temporal asociada al proceso de cuantización es aproximadamente del orden de un período de muestreo.

De esta manera, al aumentar (fs), se obtiene un menor período (Ts), aumentando la resolución temporal de la medición. La guía diferencia esta incertidumbre de cuantización del *jitter* físico del transmisor.

III. DESARROLLO DE LA PRACTICA

La práctica se desarrolló con el propósito de analizar experimentalmente una comunicación serial UART utilizando una Raspberry Pi Pico 2W y un analizador lógico mediante el software Logic 2. El procedimiento se realizó inicialmente configurando el microcontrolador para generar una señal UART conocida y posteriormente capturando dicha señal para estudiar su estructura, su decodificación y la influencia de la frecuencia de muestreo sobre la medición del tiempo de bit.

En la primera etapa se configuró la Raspberry Pi Pico 2W utilizando MicroPython. Para la comunicación serial se empleó el UART 0, configurado inicialmente a una velocidad de 9600 baudios, con 8 bits de datos, sin paridad y un bit de parada. La transmisión se realizó mediante el pin GPIO 0, correspondiente a la salida TX del UART. Como parte de la programación inicial, se configuró el microcontrolador para transmitir periódicamente el carácter “U”, generando una trama serial que pudiera ser observada fácilmente mediante el analizador lógico. Esta configuración permitió disponer de una señal repetitiva y estable para realizar las primeras mediciones.

Posteriormente, se realizó la conexión entre la Raspberry Pi Pico 2W y el analizador lógico. El canal utilizado para la

captura se conectó al pin GPIO 0, correspondiente a la señal TX, mientras que la referencia de tierra del analizador se conectó a GND de la Raspberry Pi Pico 2W. De esta manera, el analizador pudo registrar directamente la señal UART de nivel lógico generada por el microcontrolador. Es importante señalar que en esta práctica se analizó directamente una señal UART de aproximadamente 3,3 V y no una interfaz RS232 convencional, ya que esta última utiliza niveles eléctricos diferentes y requiere una etapa de conversión para poder conectarse directamente a un dispositivo de lógica digital.

Una vez realizada la conexión física, se inició el software Logic 2 y se configuró el canal correspondiente para realizar la captura de la señal digital. Sobre la captura obtenida se incorporó el analizador de protocolo Async Serial, utilizando inicialmente los mismos parámetros configurados en la Raspberry Pi Pico 2W: una velocidad de 9600 baudios, 8 bits de datos, ausencia de paridad y un bit de parada. Esta configuración permitió que Logic 2 interpretara los cambios de nivel de la señal como una comunicación serial UART y mostrara la información transmitida de manera organizada.

Con la señal correctamente capturada se procedió a observar la trama UART en diferentes representaciones. El objetivo fue comprobar que los mismos datos transmitidos podían ser interpretados en formato binario, decimal, hexadecimal y ASCII. Para realizar esta comprobación se utilizaron diferentes instancias del analizador Async Serial sobre la misma captura. Esto permitió relacionar los cambios de nivel observados físicamente en la señal con los valores digitales correspondientes a los caracteres transmitidos.

A continuación, se modificó el programa de la Raspberry Pi Pico 2W para transmitir una secuencia de caracteres en lugar de un único carácter. Como mensaje de prueba se utilizó “UMNG_2026_LIDER_EN_TELECOMUNICACIONES”. La secuencia transmitida fue capturada nuevamente mediante Logic 2 y posteriormente decodificada utilizando el analizador Async Serial. De esta manera se comprobó que la información transmitida por el microcontrolador podía ser recuperada correctamente a partir de la señal digital capturada por el analizador lógico.

Después de comprobar el funcionamiento de la comunicación, se realizó el análisis de sensibilidad a la frecuencia de muestreo. Para esta parte de la práctica se utilizó la configuración de comunicación indicada en la guía, correspondiente a 9600 baudios, 8 bits de datos, paridad impar y un bit de parada. A partir de esta configuración se realizaron diferentes capturas de la misma señal, modificando únicamente la frecuencia de muestreo del analizador lógico. Las frecuencias consideradas fueron (25 kS/s, 50 kS/s, 100 kS/s, 200 kS/s, 250 kS/s, 500 kS/s, 1 MS/s, 2 MS/s, 4 MS/s, 8 MS/s, 12 MS/s, 16 MS/s y 24 MS/s).

Para cada frecuencia de muestreo se observó la señal obtenida en Logic 2 y se midió experimentalmente el tiempo correspondiente a un bit utilizando las herramientas de

medición temporal del programa. Los valores obtenidos fueron registrados en la tabla de resultados. Debido a que el analizador lógico realiza la captura en instantes discretos determinados por su frecuencia de muestreo, los tiempos de bit medidos experimentalmente presentaron algunas diferencias respecto al valor teórico. Estas diferencias fueron conservadas en la tabla, ya que corresponden a las condiciones reales de la medición y permiten analizar el efecto de la resolución temporal del instrumento.

El tiempo de bit teórico se determinó a partir de la relación entre la tasa de transmisión y la duración de cada bit:

$$T_b = \frac{1}{f_b}$$

Para una transmisión de 9600 baudios, por ejemplo, el tiempo de bit teórico es:

$$T_b = \frac{1}{9600} = 104,17\mu s$$

Este valor fue utilizado como referencia para comparar las mediciones obtenidas experimentalmente. A frecuencias de muestreo bajas se observaron diferencias mayores respecto al valor teórico, mientras que al incrementar la frecuencia de muestreo los valores medidos tendieron a aproximarse a los valores esperados.

También se determinó el número de muestras disponibles para representar cada bit de la señal. Para ello se utilizó la relación:

$$N = \frac{f_s}{f_b}$$

donde N representa el número de muestras por bit, f_s corresponde a la frecuencia de muestreo y f_b representa la tasa de transmisión. Este cálculo permitió evaluar de manera cuantitativa la capacidad del analizador lógico para representar temporalmente la señal UART.

Como criterio de análisis se utilizó un mínimo de diez muestras por bit. Por tanto, cuando el resultado de la relación anterior fue mayor o igual a diez, se consideró que la condición de muestreo cumplía el criterio establecido para la práctica. En caso contrario, se marcó la condición como no satisfactoria. Este procedimiento se aplicó a las diferentes tasas de transmisión consideradas en la tabla experimental: 1200, 9600, 57600 y 115200 baudios.

Diseño Electrónico.

El montaje experimental estuvo conformado por una Raspberry Pi Pico 2W, un analizador lógico y un computador con el software Logic 2. La Raspberry Pi Pico 2W se utilizó como dispositivo transmisor de la comunicación serial UART, generando la señal digital que posteriormente fue capturada y analizada. Para la práctica, la comunicación UART se configuró con una velocidad de transmisión de 9600 baudios, 8

bits de datos, paridad impar y un bit de parada, de acuerdo con la configuración establecida para el análisis de sensibilidad a la frecuencia de muestreo.

La señal de transmisión se obtuvo desde el pin GPIO 0, correspondiente a la salida TX del UART de la Raspberry Pi Pico 2W, y se conectó al canal CH1 del analizador lógico. Además, se realizó una conexión entre el terminal GND de la Raspberry Pi Pico 2W y el terminal GND del analizador lógico, estableciendo así una referencia común para la medición. De esta manera, el analizador pudo registrar los cambios de nivel lógico producidos por la transmisión de los datos.

El analizador lógico se conectó mediante USB al computador en el que se ejecutó Logic 2. Dentro del software se configuró el analizador de protocolo **Async Serial**, utilizando los mismos parámetros de comunicación establecidos en el microcontrolador. La señal capturada en el canal CH1 fue posteriormente decodificada por Logic 2, permitiendo observar la información transmitida y analizarla mediante diferentes representaciones, como ASCII, hexadecimal y binaria.

Este montaje permitió estudiar directamente la señal UART generada por la Raspberry Pi Pico 2W y analizar experimentalmente cómo la frecuencia de muestreo del analizador lógico afecta la representación temporal de la señal y la cantidad de muestras disponibles para cada bit.

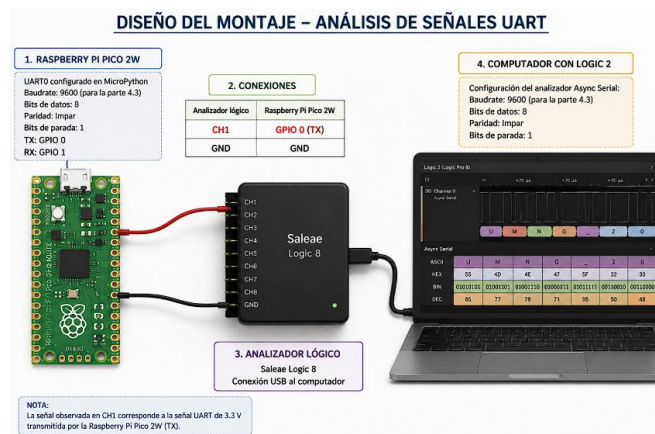


Ilustración 3. montaje de la practica

Desarrollo del Software.

Envío de caracteres ASCII.

En el primer ejercicio se programó la Raspberry Pi Pico 2W mediante MicroPython para realizar una comunicación UART. Se configuró el UART 0 a 9600 baudios, con 8 bits de datos, sin paridad y un bit de parada. La transmisión se realizó por el GPIO 0 (TX), enviando periódicamente el carácter “U” para facilitar la identificación de la trama.


```
from machine import Pin, UART
import utime

uart = UART(0, baudrate=9600, bits=8, parity=None, stop=1,
            tx=Pin(0), rx=Pin(1))

while True:
    uart.write("U")
    utime.sleep(1)
```

Ilustración 4. programa en thonny, micropython

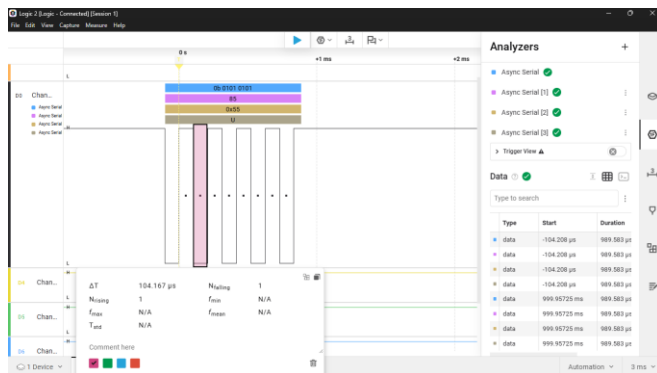


Ilustración 5. envío de carácter ascii

En el análisis de logic 2, primero se configuro el programa para los valores del carácter y se estableció una frecuencia de muestreo de 24Mhz, con esto se pudo observar que el tiempo de bit es totalmente semejante al valor teórico del tiempo de bit con los 9600 baudios 104,167µs

Análisis de la trama UART

En el segundo ejercicio se conectó el GPIO 0 de la Raspberry Pi Pico 2W al canal CH1 del analizador lógico y se conectaron ambas tierras. La señal fue capturada en Logic 2 y decodificada mediante el analizador Async Serial. Se observaron los datos en formato binario, decimal, hexadecimal y ASCII, comprobando la correspondencia entre la señal digital y el carácter transmitido.

Posteriormente se modificaron los datos enviados y se transmitieron varios caracteres consecutivos. Finalmente, se envió el mensaje "UMNG_2026_LIDER_EN_TELECOMUNICACIONES", realizando una nueva captura para analizar la trama completa y medir su duración.

```
from machine import Pin, UART
import utime

uart = UART(0, baudrate=9600, bits=8, parity=1, stop=1,
            tx=Pin(0), rx=Pin(1))

mensaje = "UMNG_2026_LIDER_EN_TELECOMUNICACIONES"

while True:
    uart.write(mensaje)
    utime.sleep(1)
```

Ilustración 6. programa en thonny, micropython

Calculamos el tiempo del envío de la trama para comparar con el tiempo que se pueda analizar en logic 2.

$$T = \frac{37 \times 11}{9600} = \frac{407}{9600} = 42.4ms$$

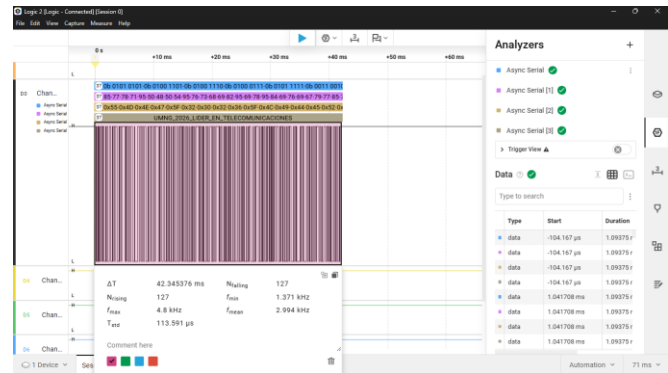


Ilustración 7. análisis de trama de bits

Como se puede percibir en la señal el tiempo de envío de la trama de bits coincide con el valor teórico, por otro lado, se puede observar los datos enviados en diferentes formatos.

Para realizar esta medición se emplearon los Timing Markers de Logic 2, ubicando un cursor al inicio y otro al final del bit de interés, lo que permitió leer directamente la duración Tb sobre la señal capturada y compararla con el valor teórico calculado.

Adicionalmente, se activó la opción Binary del analizador Async Serial y se amplió la escala temporal de la señal capturada, lo que permitió observar con mayor detalle los bits de inicio, datos y parada dentro de cada trama, confirmando visualmente la estructura del protocolo UART configurado.

Análisis de la frecuencia de muestreo

En el tercer ejercicio se estudió el efecto de la frecuencia de muestreo sobre la captura de la señal UART. Se realizaron diferentes capturas modificando únicamente fs, utilizando frecuencias entre 25 kS/s y 24 MS/s. Para cada caso se registró el tiempo de bit y se calculó el número de muestras por bit mediante:

$$N = \frac{fs}{fb}$$

Los resultados se compararon con el criterio establecido de mínimo 10 muestras por bit. Se observó que, al aumentar la frecuencia de muestreo, aumenta el número de muestras disponibles y mejora la resolución temporal de la señal.

A modo de ejemplo, el siguiente código corresponde a la configuración empleada para la tasa de 115200 baudios; para las demás tasas evaluadas (1200, 9600 y 57600 baudios) únicamente se modificó el parámetro baudrate del objeto UART, manteniendo el resto del programa sin cambios.

```
from machine import Pin, UART
import utime

uart = UART(0, baudrate=115200, bits=8, parity=1, stop=1,
            tx=Pin(0), rx=Pin(1))

mensaje = "UMNG_2026_LIDER_EN_TELECOMUNICACIONES"

while True:
    uart.write(mensaje)
    utime.sleep(1)
```

Ilustración 8. programa en thony para analisis de muestro

Utilizamos el mismo programa, tan solo modificamos los valores de baudios, para la práctica.

En esta etapa de la práctica se analizó experimentalmente el efecto de la frecuencia de muestreo sobre la captura y medición de una señal UART. Para ello, se realizaron diferentes capturas modificando la frecuencia de muestreo del analizador lógico y manteniendo las condiciones de transmisión establecidas. Los resultados obtenidos para cada condición se presentan en la tabla experimental, donde se relacionan la frecuencia de muestreo, el tiempo de bit medido, el número de muestras por bit y el cumplimiento del criterio establecido.

Teniendo en cuenta que son diferentes frecuencias de muestro y valor de baudios, entonces es muy importante modificar los parámetros.

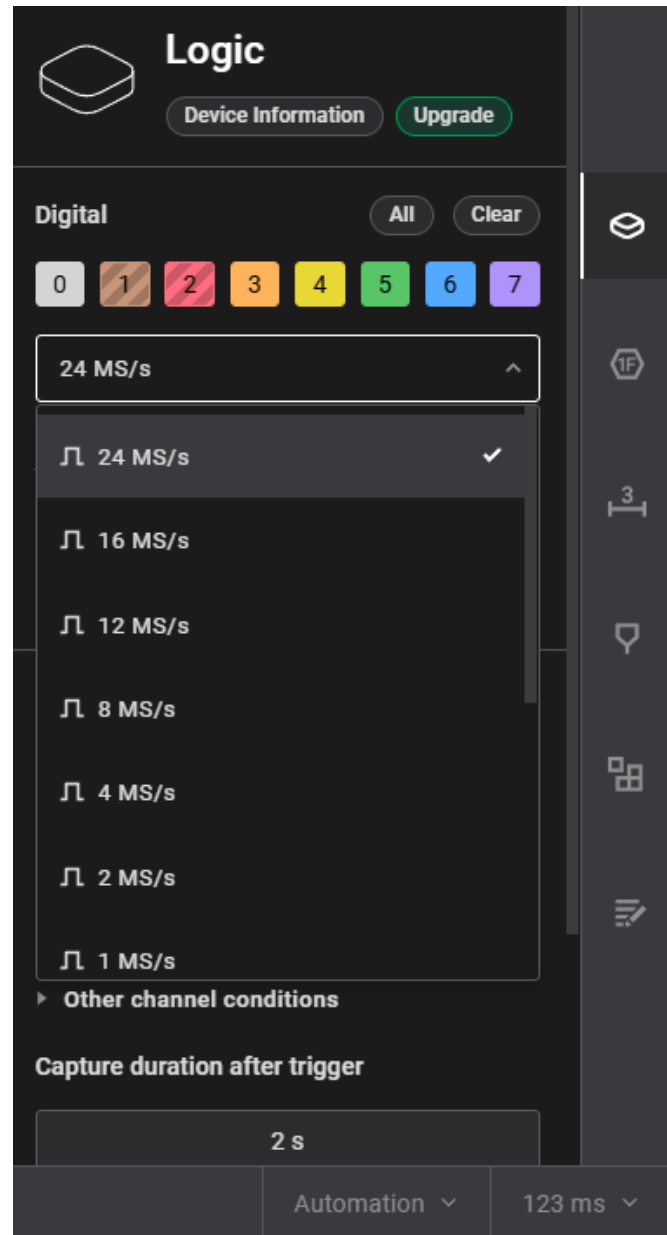


Ilustración 9. apartado de configuración de frecuencia de muestreo

En este apartado modificamos los valores de frecuencia de muestreo, el canal en que se desea observar y el tiempo de duración después del disparo de la señal.

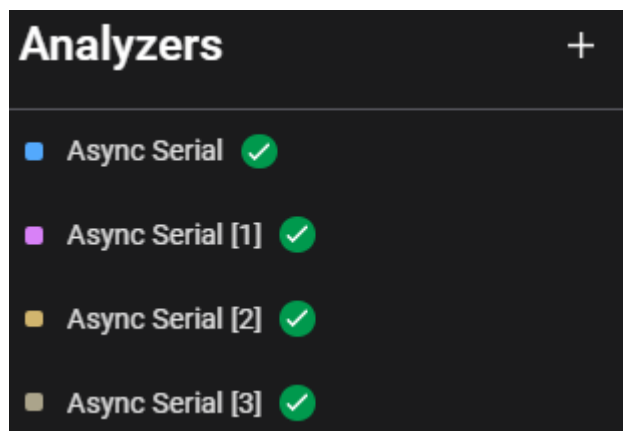


Ilustración 10. apartado de analizadores

En este caso el logic 2 nos permite agregar analizadores del puerto con diferentes formatos como ASSCI, Hexadecimal, decimal y binario.

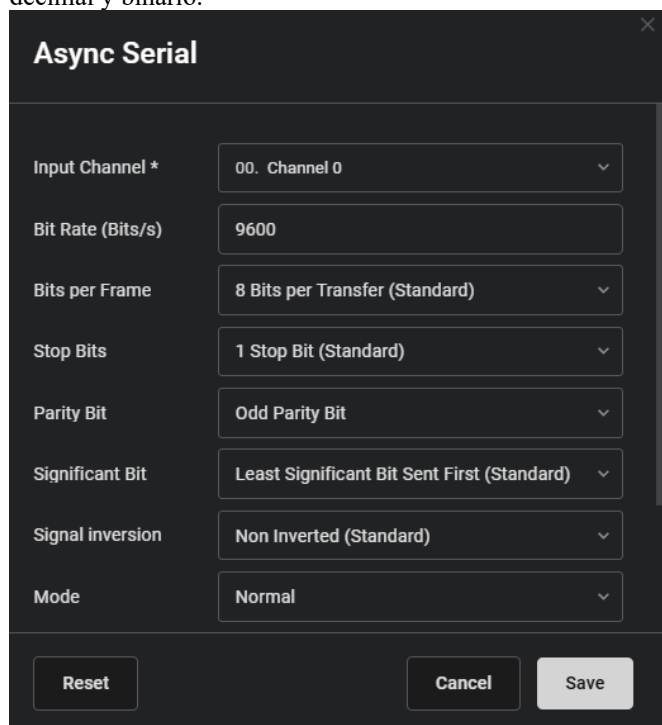


Ilustración 11. apartado de configuración de parámetros

Es muy importante configurar estos analizadores con los parámetros de la señal, en este apartado se configuran diferentes parámetros importantes.

Para una tasa de transmisión de **1200 baudios**, se observa que todas las frecuencias de muestreo utilizadas permiten obtener más de 10 muestras por bit. Esto se debe a que el tiempo de duración de cada bit es relativamente grande, por lo que incluso las frecuencias de muestreo más bajas proporcionan suficiente información para representar correctamente la señal. Además, a medida que aumenta la frecuencia de muestreo, los tiempos

medidos presentan una mayor estabilidad alrededor del valor teórico.

En **9600 baudios** se observa con mayor claridad la influencia de la frecuencia de muestreo. Las primeras condiciones de la tabla no alcanzan el mínimo de muestras requerido, por lo que la representación de la señal resulta menos precisa. A partir de **100 kS/s** se alcanza el criterio establecido y, conforme se incrementa la frecuencia de muestreo, las mediciones experimentales se aproximan cada vez más al tiempo de bit teórico. Esto evidencia que una mayor cantidad de muestras permite determinar con mayor precisión los cambios de nivel de la señal.

Para **57600 baudios**, el tiempo de cada bit es considerablemente menor, por lo que las frecuencias de muestreo bajas proporcionan muy pocas muestras para representar cada intervalo. En la tabla se observa que las condiciones iniciales no cumplen el criterio establecido y que este comienza a cumplirse a partir de **1 MS/s**. A partir de este punto, la señal puede analizarse con una resolución temporal considerablemente mejor.

El comportamiento más exigente se presenta a **115200 baudios**, debido a que corresponde a la mayor velocidad de transmisión analizada. En este caso, las frecuencias de muestreo inferiores a **2 MS/s** no proporcionan las suficientes muestras por bit para cumplir el criterio de la práctica. A partir de 2 MS/s la cantidad de muestras aumenta considerablemente y permite obtener una representación más adecuada de la señal. Esto demuestra que, cuando aumenta la velocidad de transmisión, también debe aumentar la frecuencia de muestreo necesaria para realizar una medición confiable.

Al comparar los resultados experimentales de las cuatro tasas de transmisión se puede observar una tendencia clara: **a mayor velocidad de transmisión, mayor frecuencia de muestreo se necesita para representar adecuadamente cada bit**. Esto ocurre porque al aumentar la velocidad de transmisión disminuye la duración de cada bit y, por lo tanto, se requiere una adquisición más rápida para conservar suficientes muestras durante dicho intervalo.

También se evidencia que las diferencias entre los valores teóricos y experimentales son más notorias cuando se utilizan frecuencias de muestreo bajas. Estas diferencias son producto de la resolución temporal limitada del proceso de adquisición y de la forma en que el analizador lógico determina los instantes de transición de la señal. Al incrementar la frecuencia de muestreo, las mediciones tienden a acercarse al comportamiento teórico, tal como se observa en los resultados registrados en la tabla.

Tabla 1: Análisis de muestras por bit según la tasa de transmisión y la frecuencia de muestreo

Baudios / f s	250	500	1000	2000	2500	5000	1M	2M	4M	8M	12M	16M	24M
1200	Tasa de baudios de referencia: 1200 baudios (informático, el cálculo real usa el tiempo de bit que ingresa abajo, columna por columna)												
Tiempo de bit (µs)	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00	840.00
Muestras por bit (N)	21.00	42.00	83.00	166.00	209.00	416.00	833.00	1667.00	3333.40	6666.96	10000.92	13333.92	20000.88
% ≥ 10	x	x	x	x	x	x	x	x	x	x	x	x	x
5000	Tasa de baudios de referencia: 5000 baudios (informático, el cálculo real usa el tiempo de bit que ingresa abajo, columna por columna)												
Tiempo de bit (µs)	120.00	100.00	100.00	100.00	104.00	104.00	104.00	104.00	104.25	104.12	104.17	104.18	104.17
Muestras por bit (N)	3.00	5.00	10.00	21.00	26.00	52.00	104.00	208.00	417.00	832.96	1250.00	1666.88	2500.00
% ≥ 10	x	x	x	x	x	x	x	x	x	x	x	x	x
57600	Tasa de baudios de referencia: 57600 baudios (informático, el cálculo real usa el tiempo de bit que ingresa abajo, columna por columna)												
Tiempo de bit (µs)	20.00	20.00	20.00	15.00	16.00	16.00	17.00	17.50	17.25	17.38	17.33	17.38	17.38
Muestras por bit (N)	0.50	1.00	2.00	3.00	4.00	9.00	17.00	35.00	69.00	139.00	207.96	276.00	417.00
% ≥ 10	x	x	x	x	x	x	x	x	x	x	x	x	x
115200	Tasa de baudios de referencia: 115200 baudios (informático, el cálculo real usa el tiempo de bit que ingresa abajo, columna por columna)												
Tiempo de bit (µs)	40.00	20.00	10.00	10.00	8.00	8.00	8.00	8.50	8.50	8.75	8.66	8.62	8.70
Muestras por bit (N)	1.00	1.00	1.00	2.00	2.00	4.00	8.00	17.00	34.00	70.00	103.92	137.92	208.00
% ≥ 10	x	x	x	x	x	x	x	x	x	x	x	x	x

• Celdas amarillas = tiempo de bit a ingresar (µs), una por cada frecuencia de muestreo

Nota: para cada frecuencia de muestreo (columna) ingrese el tiempo de bit (µs) en la celda amarilla correspondiente. Con ese dato se calcula la tasa de baudios efectiva de esa columna ($f_B = 1 / 000\ 000 / T_{bit}$), luego las muestras por bit ($N = f_s \cdot T_{bit}$ o $N = T_{bit} / f_B$) y si $N \geq 10$.

Ilustración 12. imagen que corresponde a la tabla de la practica

Por otra parte, utilizar una frecuencia de muestreo excesivamente alta no siempre representa la mejor opción. Aunque permite obtener una mayor cantidad de información sobre la señal, también genera un volumen mayor de datos durante la captura. Por esta razón, la frecuencia de muestreo debe seleccionarse de acuerdo con la velocidad de la comunicación y con el nivel de precisión requerido para el análisis.

¿Qué efecto tiene el cambio en la tasa de muestreo para determinar el tiempo de bit?

El cambio en la tasa de muestreo afecta directamente la precisión con la que se puede determinar el tiempo de bit. Cuando la frecuencia de muestreo es baja, se dispone de pocas muestras para representar cada bit, por lo que la medición puede presentar mayores diferencias respecto al valor teórico. Al aumentar la frecuencia de muestreo, se obtienen más muestras dentro de cada bit y, por tanto, una representación temporal más precisa de la señal. Esto se observó en la práctica, especialmente al trabajar con velocidades de transmisión altas como 57600 y 115200 baudios, donde fue necesario utilizar frecuencias de muestreo mayores para obtener una cantidad suficiente de muestras por bit.

¿Qué desventaja operativa o de almacenamiento puede presentarse al configurar tasas de muestreo excesivamente altas, como 24 MS/s, durante capturas de larga duración?

Utilizar una tasa de muestreo excesivamente alta, como 24 MS/s, genera una gran cantidad de muestras durante la captura. Aunque esto proporciona una mayor resolución temporal, también aumenta considerablemente el volumen de datos que debe almacenar y procesar el analizador lógico. En capturas de larga duración, esto puede provocar un mayor consumo de memoria, archivos de captura más grandes y un procesamiento más lento. Por esta razón, no siempre es conveniente utilizar la máxima tasa de muestreo disponible; debe seleccionarse una frecuencia que proporcione la resolución necesaria sin generar datos innecesarios.

IV. RESULTADOS

Los resultados obtenidos durante la práctica permitieron comprobar experimentalmente el funcionamiento de una comunicación UART mediante la Raspberry Pi Pico 2W y su posterior captura y decodificación con el analizador lógico en Logic 2. La señal transmitida por el GPIO 0 fue identificada correctamente y pudo ser interpretada mediante el analizador Async Serial, permitiendo observar la información en diferentes representaciones y verificar la correspondencia entre los datos transmitidos y los datos recibidos.

A partir de las capturas realizadas se comprobó la estructura de las tramas UART y se logró identificar correctamente los caracteres enviados. También fue posible transmitir y recuperar el mensaje completo utilizado en la práctica, lo que permitió verificar el funcionamiento de la comunicación de extremo a extremo. Las mediciones realizadas sobre la señal permitieron obtener experimentalmente el tiempo de bit y compararlo con el valor teórico correspondiente a cada velocidad de transmisión.

El análisis de la frecuencia de muestreo permitió observar que la calidad de la medición depende directamente de la cantidad de muestras disponibles durante cada bit. Para las velocidades de transmisión más bajas, las frecuencias de muestreo utilizadas proporcionaron una cantidad suficiente de muestras, mientras que al incrementar la velocidad de transmisión fue necesario aumentar la frecuencia de muestreo para cumplir con el criterio establecido de la práctica. Esta tendencia se evidencia en la tabla de resultados presentada anteriormente.

Los valores experimentales del tiempo de bit presentaron pequeñas diferencias respecto a los valores teóricos. Estas diferencias se mantuvieron en la tabla debido a que corresponden a errores propios del proceso experimental y de la resolución temporal del analizador lógico. En general, al aumentar la frecuencia de muestreo, las mediciones obtenidas presentaron un comportamiento más cercano al esperado teóricamente.

V. CONCLUSIONES

La práctica permitió comprobar de manera experimental el funcionamiento de una comunicación UART utilizando una Raspberry Pi Pico 2W y un analizador lógico. La captura realizada mediante Logic 2 permitió identificar correctamente las tramas transmitidas y relacionar la señal digital observada con los datos representados en diferentes formatos.

El análisis realizado permitió comprobar que la frecuencia de muestreo tiene una influencia directa sobre la precisión con la que puede analizarse una señal UART. A medida que aumenta la velocidad de transmisión, disminuye la duración de cada bit y, por lo tanto, es necesario utilizar una mayor frecuencia de muestreo para disponer de suficientes muestras y representar adecuadamente la señal.

Los resultados obtenidos también demostraron que las diferencias entre los valores teóricos y experimentales son normales dentro del proceso de medición. Estas diferencias fueron más evidentes en las condiciones de menor frecuencia de muestreo, mientras que al aumentar la frecuencia de adquisición se obtuvo una representación temporal más precisa. Finalmente, se comprobó que la selección de la frecuencia de muestreo debe realizarse considerando tanto la velocidad de transmisión como la resolución requerida para el análisis. Una frecuencia demasiado baja puede dificultar la correcta interpretación de la señal, mientras que una frecuencia excesivamente alta genera una mayor cantidad de datos sin que necesariamente sea indispensable para todas las condiciones de transmisión. En conjunto, la práctica permitió relacionar los conceptos teóricos de comunicación UART y muestreo digital con mediciones realizadas directamente sobre una señal real.

<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>

VI. BIBLIOGRAFIA

[1] Raspberry Pi Ltd, *Raspberry Pi Pico 2 W Datasheet: An RP2350-based microcontroller board with wireless functionality*, Cambridge, UK: Raspberry Pi Ltd, 2024. [En línea]. Disponible: <https://datasheets.raspberrypi.com/picow/pico-2-w-datasheet.pdf>

[2] Saleae, Inc., "Async Serial Analyzer User Guide," *Saleae Logic 2 Software Documentation*, San Francisco, CA, EE. UU., 2024. [En línea]. Disponible: <https://support.saleae.com/protocol-analyzers/analyzer-user-guides/using-async-serial>

[3] Analog Devices, "UART: A Hardware Communication Protocol: Understanding Universal Asynchronous Receiver/Transmitter," *Analog Dialogue / Technical Resources*, EE. UU., 2023. [En línea]. Disponible: <https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html>

[4] B. Sklar y F. Harris, *Digital Communications: Fundamentals and Applications*, 3a ed., Boston, MA, EE. UU.: Pearson / Prentice Hall, 2021, cap. 3, pp. 115–142.

[5] J. W. Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, 6a ed., Austin, TX, EE. UU.: Jonathan W. Valvano, 2020, cap. 7, pp. 312–335.

[6] Keysight Technologies, *Evaluating Digital Logic Analyzers and Oscilloscope Timing Resolution*, Application Note 5989-2111EN, Santa Clara, CA, EE. UU., 2022

[7] Raspberry Pi Foundation, "UART Configuration," *Raspberry Pi Documentation*, Cambridge, UK, 2024. [En línea]. Disponible:

